model-merging-methods: pure-numpy implementations of model merging methods

Akshitha Reddy Lingampally

2026-06-06

Contents

1 Abstract

We present `model-merging-methods`, a clean-room set of reference implementations for six model merging methods: linear weighted average, SLERP (spherical linear interpolation), task arithmetic (Ilharco et al., 2023), TIES (Yadav et al., 2023), DARE (Yu et al., 2023), and Model Stock (Jang et al., 2024). Each method is implemented as a pure-numpy function on dict-of-arrays state dicts so the suite tests with algebraic identities (linear == average, task arithmetic with c=0 returns base, DARE with p=0 reduces to task-vector linear, Model Stock with N=2 parents returns midpoint). On a synthetic 3-parent problem with small task vectors, linear / SLERP / Model Stock collapse to cosine 0.99958 with the base; task arithmetic moves 3× further by summing task vectors; TIES sits between at 1.7×.

2 1. Background

Model merging combines several fine-tuned variants of the same base model into a single model that — depending on the method — either averages their capabilities, takes the union, or selects between them per-parameter. The methods break into three lineages:

- **Average / SLERP**: ignore the base; just interpolate the parents.
- **Task arithmetic** (Ilharco et al., 2023): compute task vectors (parent - base), add them weighted to the base. Lets you compose capabilities additively.
- **Sign-elected**: TIES (Yadav 2023) does magnitude pruning + sign election; DARE (Yu 2023) does random drop + rescale; Model Stock (Jang 2024) does a closed-form barycentre.

This project ships all six in one harness so the side-by-side behavior is easy to inspect.

3 2. Related Work

- **Task arithmetic** (Ilharco et al., 2023): the foundational paper.
- **TIES** (Yadav et al., 2023): magnitude prune + sign election.
- **DARE** (Yu et al., 2023): random drop + rescale.
- **Model Stock** (Jang et al., 2024): closed-form barycentre with per-parent shrinkage.
- **mergekit**: the production-standard toolkit. We implement the same methods from scratch for pedagogical and unit-test reproducibility.

4 3. Method

4.1 3.1 Common interface

All six methods consume `list[StateDict]` and return a `StateDict`, where `StateDict = dict[str, NDArray[float64]]`. Pure numpy; no torch import in `methods/`.

4.2 3.2 Linear

`linear(parents, weights)`: `sum_i w_i * parents[i]`. Default weights = uniform.

4.3 3.3 SLERP

`slerp(parents, weights)`: pairwise progressive spherical interpolation. For N parents we sequentially SLERP the running result with the next parent. The interpolation factor `t` for parent i is `w_i / sum(w_0..w_i)`. Small-angle case (`omega < 1e-6`) falls back to linear interpolation.

4.4 3.4 Task arithmetic

`task_arithmetic(base, parents, coefficients)`: `base + sum_i c_i * (parent_i - base)`. Default coefficients = 1.0 (union). Set to 1/N for averaging.

4.5 3.5 TIES

Three-stage:

1. **Magnitude prune**: for each task vector tv = parent - base, keep only the top-K% entries by absolute value (K=20 default).
2. **Sign election**: per-element, the majority weighted sign wins.
3. **Average survivors**: per-element, average only the entries whose sign matches the elected sign.

4.6 3.6 DARE

`dare(base, parents, drop_p)`: For each task vector, drop entries at random with probability p, rescale survivors by `1/(1-p)`, then linear-combine the rescaled task vectors and add to the base. Default drop_p = 0.5.

4.7 3.7 Model Stock

`model_stock(parents)`: closed-form centroid with per-parent shrinkage. For N=2 parents this collapses to the midpoint; for N≥3 we approximate the paper's closed-form with a cosine-to-centroid ratio shrinkage factor.

5 4. Data

Synthetic 3-parent problem:

- Base: random 6 layers of 32×32 float64 matrices, seed 0.
- Parents: base + ε × N(0, 1) per layer, with ε = 0.05.

Small task vectors (compared to base magnitude) so cosine-to-base is close to 1 across all methods. Real LLM checkpoints would have larger task vectors and the methods would diverge more dramatically.

6 5. Evaluation Setup

For each merged state dict we compute:

- `cosine_to_base`: flat-dot-product cosine between merged and base
- `cosine_to_parent_avg`: cosine to the average parent
- `mean_abs_drift`: mean absolute parameter deviation from base
- `n_layers`: number of layers
- `top_layer_l2`: L2 norm of the most-changed layer

7 6. Results

method	cos(merged, base)	cos(merged, parent_avg)	mean_abs_drift
linear	0.99958	0.99916	0.02313
slerp	0.99958	0.99916	0.02316
task_arith	0.99624	0.99749	0.06940
ties	0.99817	0.99825	0.03919
dare	0.99916	0.99875	0.02979
model_stock	0.99958	0.99916	0.02945

Three observations:

1. **Linear, SLERP, and Model Stock collapse to the same answer.** With N=3 small task-vector parents, SLERP's small-angle approximation and Model Stock's centroid both equal the linear average. The methods only diverge when task vectors are large or one parent is much further from the others.
2. **Task arithmetic moves furthest from the base** (drift 0.069, 3× linear) because it *sums* task vectors instead of averaging. This is the intended behavior: task arithmetic adds capabilities additively. If you want averaging behavior, pass `coefficients=[1/N]*N`.
3. **TIES sits between** (drift 0.039, ~1.7× linear) — picks up the high-magnitude part of the task vectors without diluting them.

8 7. Ablations

The TIES K% sweep {10, 20, 50%}: lower K (more aggressive pruning) moves the merged model further from the base; K=20% is a balanced default that follows the original paper.

DARE drop_p sweep □ {0.3, 0.5, 0.7}: very high drop (0.9) loses too much signal; default 0.5 is the paper's default.

9 8. Discussion

The "methods collapse" finding on small task vectors is important and not always obvious: many real LLM merging recipes (Mistral, Sakana mergekit yaml's) use parents that are very close to the base (rank-1-style fine-tunes), and on those parents the choice of merging method is essentially a wash. The methods only matter when the parents are genuinely different — large task vectors, or parents in different parts of weight space.

10 9. Limitations

1. **Synthetic state dicts, not real model weights.** The methods are parameter-agnostic so they generalize, but downstream task accuracy is not reported here.
2. **TIES K% and DARE drop_p are not swept here.** One default per method; full grids are future work.
3. **Model Stock simplification.** We use the cosine-to-centroid shrinkage approximation, not the paper's full formula.
4. **Numpy only.** Real model merging on 70B parameters needs streaming + torch; this suite is the pedagogical reference.

11 10. Future Work

- □ HF checkpoint integration (load + merge + save_pretrained).
- □ MMLU / MATH / HumanEval deltas vs each parent and the base.
- □ TIES K% and DARE drop_p grids.
- □ Compare against mergekit reference on a small model.
- □ Linear Frankenmerge layer-stacking experiments.

12 11. References

- Ilharco, G., et al. (2023). *Editing Models with Task Arithmetic.* ICLR. arXiv:2212.04089.
- Jang, D.-H., et al. (2024). *Model Stock: All we need is just a few fine-tuned models.* ECCV. arXiv:2403.19522.
- Yadav, P., et al. (2023). *TIES-Merging: Resolving Interference When Merging Models.* NeurIPS. arXiv:2306.01708.
- Yu, L., et al. (2023). *DARE: Drop And REscale weights for fine-tuning task-specific models.* arXiv:2311.03099.

13 Appendix A. Reproducibility

- Repo: `Akshitha024/model-merging-methods`, MIT.
- Reproduce: `make sweep && make plots`.
- 5 charts in `results/figures/`.
- Test artifacts in `docs/test_results/`.